



Chapter 9: Functions

Course 040223101 – Computer Programming for Mathematics 1

Department of Mathematics, Faculty of Applied Science
King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year
2026

Learning Objectives

Upon completing this chapter, students will be able to:

O1

Modular Programming

Explain the benefits of functions in modular programming and how they improve code organization.

O2

Defining Functions

Define functions using the `def` statement with parameters and return values using `return`.

O3

Parameters vs. Arguments & Scope

Distinguish between parameters and arguments, and understand variable scope (local vs. global).

O4

Default Values & Multiple Returns

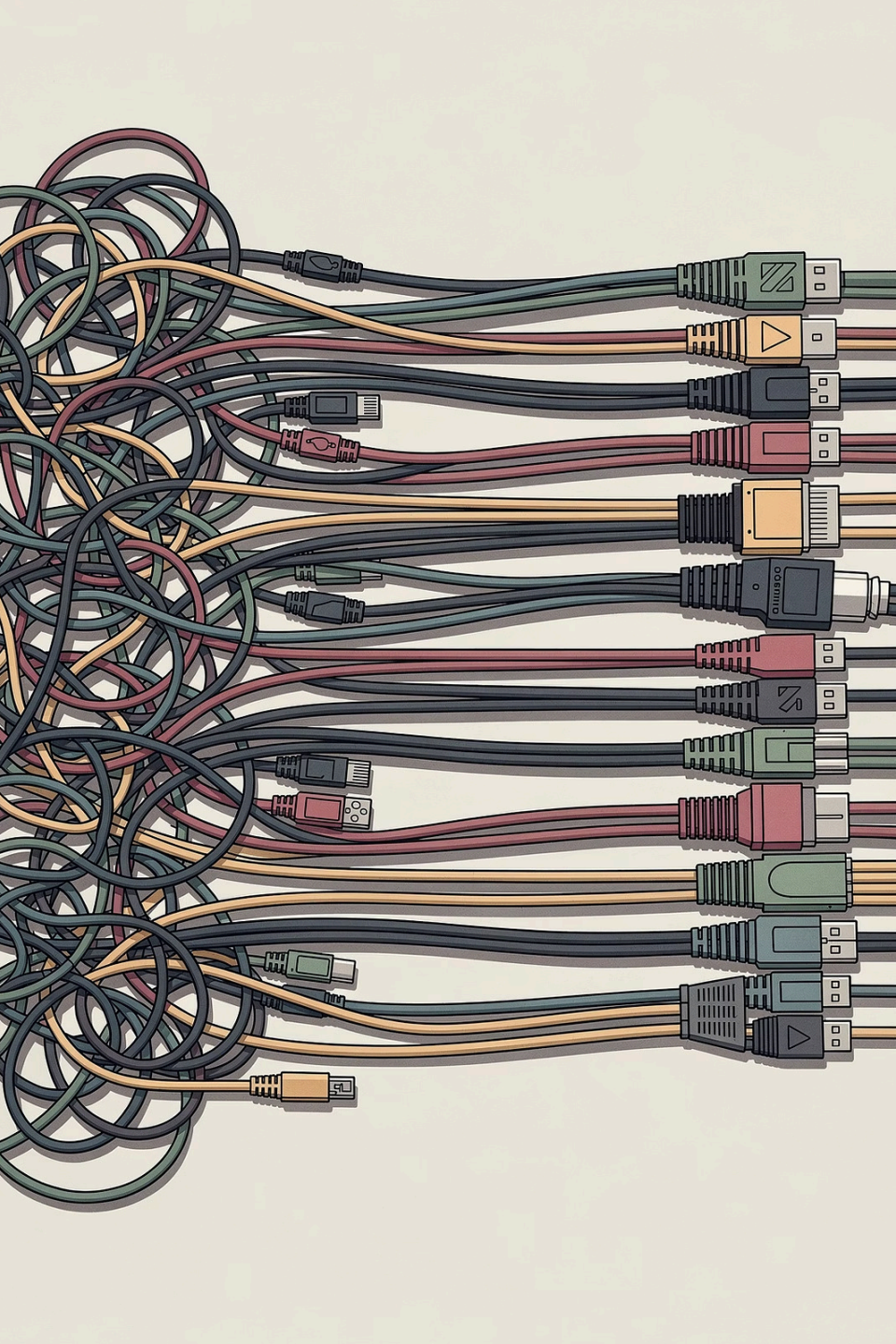
Use default parameter values and return multiple values from a single function.

O5

Recursion

Apply functions and basic recursion concepts to solve mathematical problems.

 This chapter aligns with Course Learning Outcome **CLO 4**.



Why Functions?

As programs grow larger, certain blocks of code are often needed in multiple places. Copying and pasting the same code repeatedly makes programs difficult to maintain and prone to errors. **Functions** solve this problem by grouping a set of instructions under a single name that can be called anywhere in the program.

Reusability

Write once, call many times – no need to duplicate code across the program.

Readability

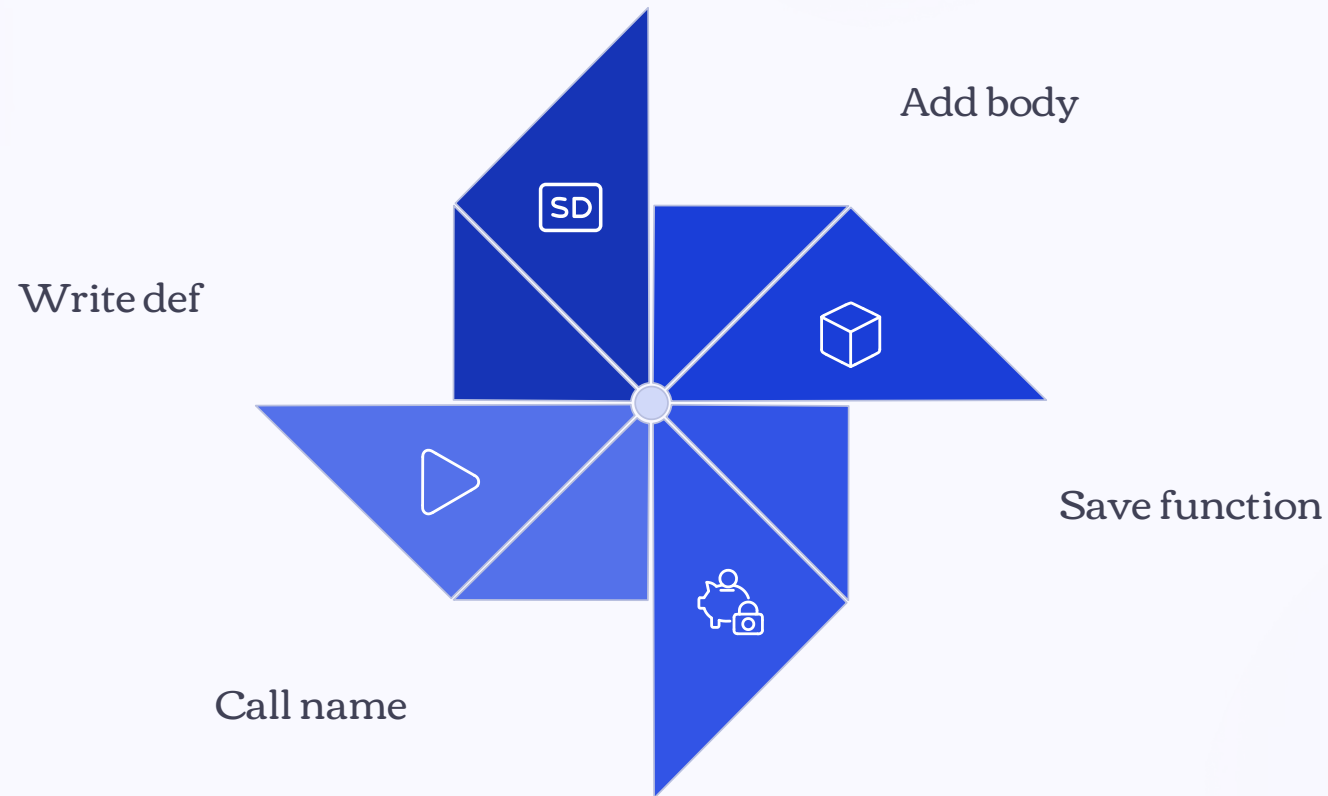
Well-named functions make code self-documenting and easier to understand at a glance.

Maintainability

Fix a bug or update logic in one place; the change propagates everywhere the function is called.

Defining and Calling Functions

A function is defined using the `def` keyword, followed by the function name and parentheses. The indented block of statements inside is the function body. To execute the function, you **call** it by writing its name followed by parentheses.



The function body runs only when the function is called – not when it is defined. The same function can be called multiple times, executing its body each time.

First Function: `greet()`

File: `09func1.py` – A simple function with no parameters that prints two lines of greeting text. It is called twice, producing the output twice.

Code

```
def greet():  
    print("สวัสดีครับ")  
    print("ยินดีต้อนรับ")  
  
greet() # Call 1  
greet() # Call 2
```

Output

```
สวัสดีครับ  
ยินดีต้อนรับ  
สวัสดีครับ  
ยินดีต้อนรับ
```

The two lines are printed **twice** – once for each call. The function body executes from top to bottom each time it is invoked.

Parameters and Arguments

Functions become far more powerful when they can accept input data. This is done through **parameters** – variable names listed inside the parentheses during the function definition. When the function is called, the actual values passed in are called **arguments**.

Parameter

The variable name declared in the `def` line. It acts as a placeholder inside the function body. Example: `name` in `def greet(name):`

Argument

The actual value supplied when calling the function. It is assigned to the parameter. Example: `"Somchai"` in `greet("Somchai")`

A function can have multiple parameters, separated by commas. Each call must supply a matching argument for each parameter (unless default values are used – covered in Section 9.4).

Parameterized Greeting: `greet(name)`

File: 09param.py – The function now accepts a `name` parameter and uses an f-string to personalize the greeting. Different arguments produce different output.

Code

```
def greet(name):  
    print(f"สวัสดีคุณ {name}")  
  
greet("Somchai")  
greet("Suda")
```

Output

```
สวัสดีคุณ Somchai  
สวัสดีคุณ Suda
```

The same function produces a different greeting for each argument. This demonstrates how parameters allow a single function definition to handle varied inputs.

SECTION 9.3

Returning Values with `return`

The `return` statement sends a result back to the caller. When Python encounters `return`, the function **terminates immediately** and the specified value is passed back. This is fundamentally different from simply printing a result:

`print()` only

Displays output to the screen. The value **cannot** be used in further calculations or stored in a variable. The function returns `None` implicitly.

`return` value

Sends the value back to the caller. It **can** be stored in a variable, passed to another function, or used in an expression for further computation.

Functions that return values are the building blocks of mathematical computation in Python, enabling chained calculations and composable logic.

Square Function: `square(x)`

File: `09return.py` – The function computes and **returns** the square of its argument. The returned value can be stored or used directly in an expression.

Code

```
def square(x):  
    return x * x  
  
result = square(5)  
print(result)  
print(square(3) + square(4))
```

Output

```
25  
25
```

`square(3) + square(4)` = 9 + 16 = **25**. This demonstrates that returned values can be used directly in arithmetic expressions – something impossible with `print()` alone.

Triangle Area: `triangle_area(base, height)`

File: 09area.py – A function with **two parameters** that computes the area of a triangle using the formula $\frac{1}{2} \times \text{base} \times \text{height}$.

Code

```
def triangle_area(base, height):  
    return 0.5 * base * height  
  
print(triangle_area(10, 6))  
print(triangle_area(7, 4))
```

Output

```
30.0  
14.0
```

For base=10, height=6: $0.5 \times 10 \times 6 = 30.0$

For base=7, height=4: $0.5 \times 7 \times 4 = 14.0$

This shows how functions encapsulate mathematical formulas for clean, reusable computation.

Default Parameter Values

Python allows you to assign a **default value** to a parameter in the function definition. If the caller does not supply an argument for that parameter, the default value is used automatically. This makes functions more flexible and convenient to call.

Syntax

Place the default value after an `=` sign in the parameter list: `def power(base, exp=2):`. Parameters with defaults must come **after** parameters without defaults.

Calling without the argument

`power(5)` – only `base` is supplied; `exp` defaults to `2`, so the result is $5^2 = 25$.

Overriding the default

`power(5, 3)` – both arguments are supplied; `exp` is overridden to `3`, so the result is $5^3 = 125$.

Power Function: `power(base, exp=2)`

File: 09default.py – The exponent parameter has a default value of 2, making squaring the most common use case while still allowing any exponent.

Code

```
def power(base, exp=2):  
    return base ** exp  
  
print(power(5))    # uses exp=2  
print(power(5, 3)) # overrides exp=3
```

Output

```
25  
125
```

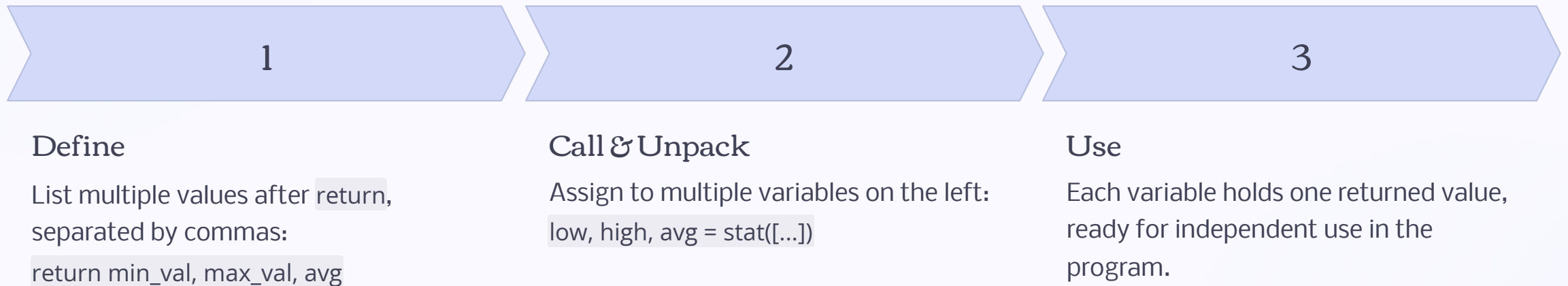
`power(5)` → $5^2 = 25$

`power(5, 3)` → $5^3 = 125$

Default values reduce the number of arguments callers must provide for common cases, making the function easier to use without sacrificing flexibility.

Returning Multiple Values

Unlike many programming languages, Python functions can return **multiple values simultaneously** by separating them with commas in the `return` statement. Python packages these values into a **tuple** behind the scenes. The caller can unpack them into separate variables.



This feature is especially useful for mathematical functions that naturally produce several related results at once, such as statistics, coordinate pairs, or quotient and remainder.

Statistics Function: `stat(numbers)`

File: 09multi.py – A single function computes and returns three statistics – minimum, maximum, and average – from a list of numbers simultaneously.

Code

```
def stat(numbers):  
    return (min(numbers),  
            max(numbers),  
            sum(numbers) / len(numbers))
```

```
low, high, avg = stat(  
    [4, 8, 15, 16, 23, 42])  
print(f"ต่ำสุด={low}, "  
      f"สูงสุด={high}, "  
      f"เฉลี่ย={avg:.2f}")
```

Output

ต่ำสุด=4, สูงสุด=42, เฉลี่ย=18.00

The list `[4, 8, 15, 16, 23, 42]` has min=4, max=42, and mean = $108/6 =$ **18.00**. All three values are returned and unpacked in a single line.

Variable Scope

Not all variables are accessible everywhere in a program. Python uses **scope** rules to determine where a variable can be seen and used.

Local Variable

Declared **inside** a function. It exists only while the function is running. When the function ends, the local variable is destroyed. Attempting to access it from outside causes a `NameError`.

Global Variable

Declared **outside** all functions. It is accessible from anywhere in the program, including inside functions (for reading).
Modifying a global variable inside a function requires the `global` keyword.

Keeping variables local prevents unintended interference between functions – a key principle of safe, modular programming. Each function operates in its own isolated namespace.

Scope in Action: 09scope.py

The variable `y` is declared inside function `f()`. It is accessible within the function but does not exist outside it.

Code

```
def f():  
    y = 10 # local variable  
    print("ในฟังก์ชัน y =", y)  
  
f()  
# print(y) # NameError!  
#         y only exists inside f()
```

Output

ในฟังก์ชัน y = 10

Inside `f()`, `y = 10` is perfectly accessible. Outside, `y` does not exist – uncommenting `print(y)` would raise a `NameError`.

⚠ Always be aware of where variables are declared. Accessing a local variable outside its function is one of the most common beginner errors.

Introduction to Recursion

A function can call **itself**. This technique is called **recursion**. It is particularly well-suited to problems that are naturally defined in terms of themselves – such as mathematical sequences and factorials.

1

Recursive Case

The function calls itself with a **simpler** or **smaller** version of the problem. Each call moves closer to the base case. Example:

$n! = n \times (n-1)!$

2

Base Case

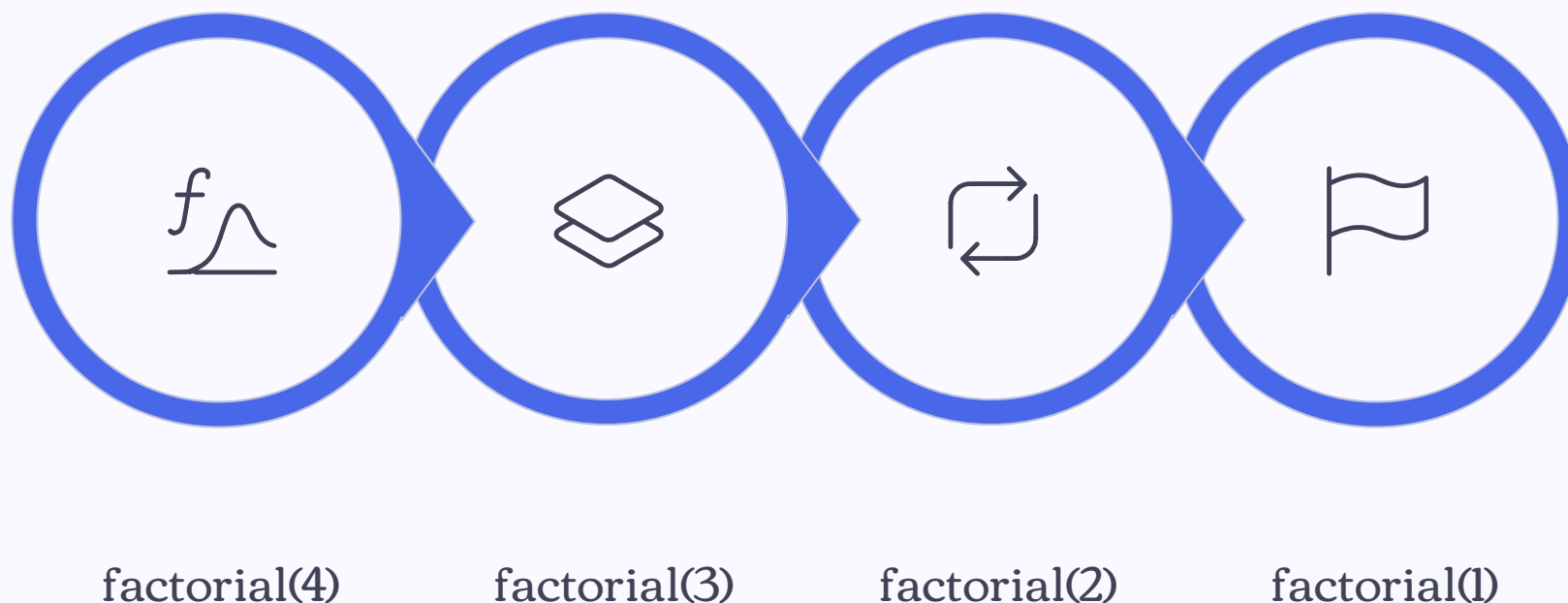
A condition that **stops** the recursion. Without it, the function calls itself forever and Python raises a `RecursionError`.

Example: $0! = 1$ and $1! = 1$

⊗ **Critical Warning:** If you forget the base case, or if the recursive step never reaches it, the recursion will never terminate and Python will raise a `RecursionError: maximum recursion depth exceeded`.

How Recursion Works: Factorial

The factorial function is the classic example of recursion. Mathematically: $n! = n \times (n - 1)!$ with base case $0! = 1! = 1$.



Each call waits for the result of the next call. Once the base case returns 1, the results multiply back up the call chain: $1 \times 2 \times 3 \times 4 = 24$. This "unwinding" is what makes recursion powerful yet potentially expensive for large inputs.

Factorial Function: `factorial(n)`

File: `09recursion.py` – A recursive implementation of the factorial function with a clear base case for `n == 0` or `n == 1`.

Code

```
def factorial(n):  
    if n == 0 or n == 1: # base case  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(5))  
print(factorial(6))
```

Output

```
120  
720
```

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$6! = 6 \times 5! = 6 \times 120 = 720$$

The base case `n == 0` or `n == 1` returns `1` immediately. For any larger `n`, the function returns `n * factorial(n-1)`, reducing the problem by one step each time.

Chapter 9 Summary



def & Calling

Define with `def`, call by name with parentheses. The body runs each time the function is called.



return

Sends results back to the caller. Enables chained calculations. Terminates the function immediately.



Scope

Local variables exist only inside their function. Global variables are accessible program-wide.



Parameters & Arguments

Parameters are placeholders in the definition; arguments are the actual values passed at call time.



Default Values

Assign defaults with `=` in the parameter list. Callers may omit those arguments.



Recursion

A function calling itself. Always requires a base case to stop. Ideal for self-referential math problems.

End-of-Chapter Exercises

The following five exercises reinforce the concepts covered in Chapter 9. Each exercise requires writing a complete Python function.

1

is_even(n)

Write a function that returns `True` if `n` is even, and `False` if `n` is odd.

2

circle(r)

Write a function that returns **both** the circumference and area of a circle with radius `r`.

3

maximum(a, b, c)

Write a function that returns the largest value among three numbers.

4

sum_to(n)

Write a function that returns the sum $1 + 2 + \dots + n$ using **recursion**.

5

fibonacci(n)

Write a function that returns the n -th term of the Fibonacci sequence using recursion.

EXERCISE 1 – SOLUTION

Solution: `is_even(n)`

This function uses the modulo operator `%` to check divisibility by 2. If the remainder is 0, the number is even; otherwise it is odd. The expression `n % 2 == 0` directly evaluates to a Boolean, so no `if` statement is needed.

Code

```
def is_even(n):  
    return n % 2 == 0  
  
print(is_even(8), is_even(7))
```

Output

```
True False
```

`8 % 2 == 0` → `True` (8 is even)

`7 % 2 == 0` → `False` (7 is odd)

The function returns a Boolean value directly – a clean, Pythonic pattern that avoids unnecessary `if/else` blocks.

EXERCISE 2 – SOLUTION

Solution: circle(r)

This function demonstrates **multiple return values**. It computes both the circumference $C = 2\pi r$ and the area $A = \pi r^2$ using $\text{pi} = 3.14159$, then returns both values as a tuple that is unpacked by the caller.

Code

```
def circle(r):  
    pi = 3.14159  
    return 2 * pi * r, pi * r ** 2  
  
c, a = circle(5)  
print(f"เส้นรอบวง={c:.2f}, "  
      f"พื้นที่={a:.2f}")
```

Output

เส้นรอบวง=31.42, พื้นที่=78.54

For $r = 5$:

$$C = 2 \times 3.14159 \times 5 = 31.42$$

$$A = 3.14159 \times 5^2 = 78.54$$

Both values are returned and unpacked in one line: `c, a = circle(5)`.

EXERCISE 4 – SOLUTION

Solution: `sum_to(n)` – Recursive Sum

This function computes $1 + 2 + \dots + n$ using recursion. The base case is `n == 1` (returning 1). The recursive case adds `n` to the sum of all integers up to `n-1`.

Code

```
def sum_to(n):  
    if n == 1:  
        return 1  
    return n + sum_to(n - 1)  
  
print(sum_to(100))
```

Output

5050

The famous result: $\sum_{k=1}^{100} k = \frac{100 \times 101}{2} = 5050$

The recursion unwinds as: `100 + sum_to(99)` → `100 + 99 + sum_to(98)` → ... → `100 + 99 + ... + 1 = 5050`.

EXERCISE 5 – SOLUTION

Solution: fibonacci(n)

The Fibonacci sequence is defined recursively: $F(n) = F(n - 1) + F(n - 2)$ with base cases $F(1) = F(2) = 1$. This is a natural fit for a recursive function.

Code

```
def fibonacci(n):  
    if n <= 2:  
        return 1  
    return fibonacci(n-1) + fibonacci(n-2)  
  
for i in range(1, 9):  
    print(fibonacci(i), end=" ")
```

Output

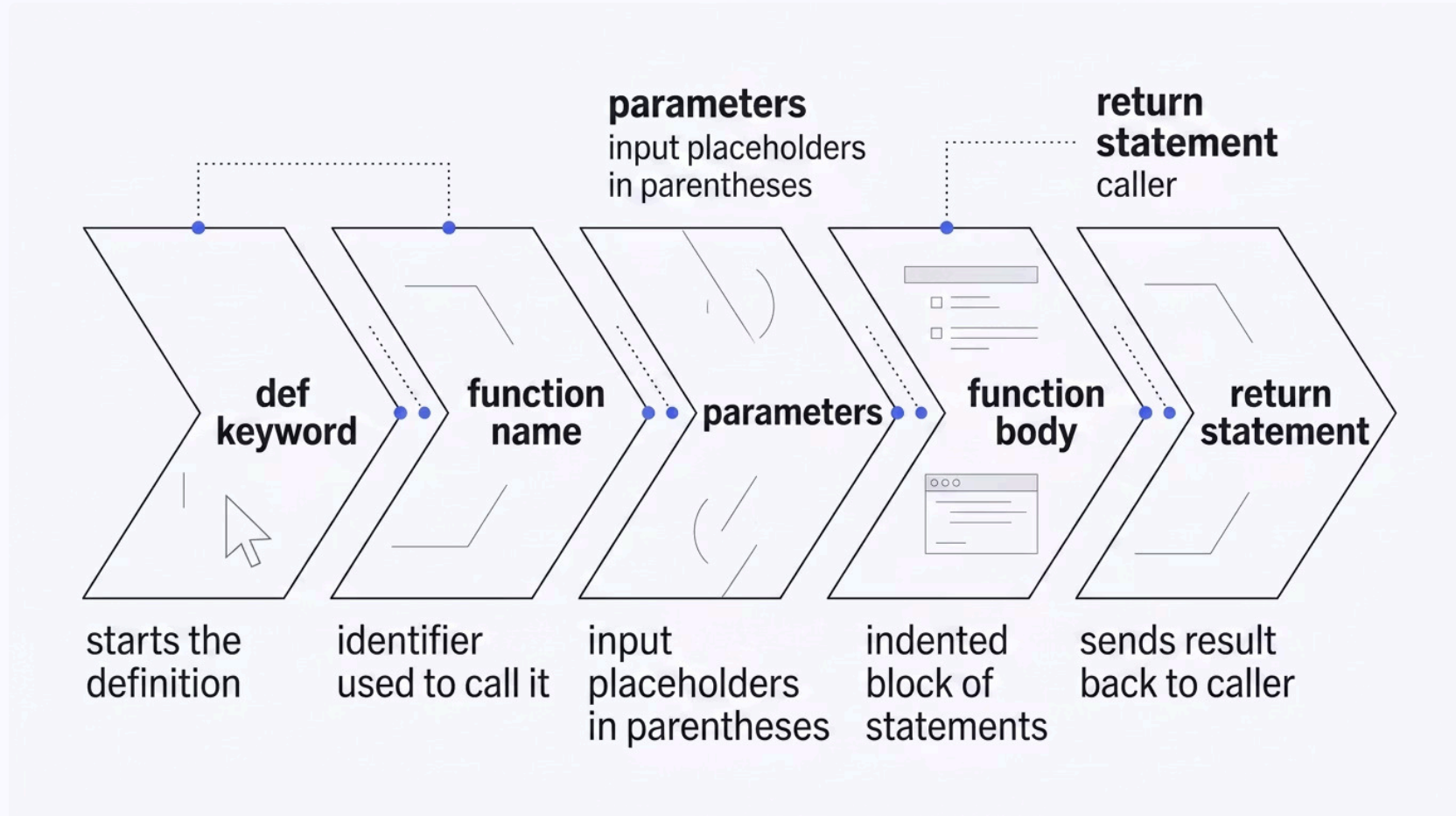
```
1 1 2 3 5 8 13 21
```

The first 8 Fibonacci numbers. Each term is the sum of the two preceding terms. The base case $n \leq 2$ returns 1 for both $F(1)$ and $F(2)$.

Note: This naive recursive implementation has exponential time complexity $O(2^n)$. For large n , memoization or iteration is preferred.

Function Anatomy: A Visual Reference

Every Python function shares the same structural components. Understanding each part is essential before writing more complex programs.



Mastering this structure allows you to read any function at a glance – identify what it accepts, what it computes, and what it returns – which is the foundation of reading and writing professional Python code.

Parameters vs. Arguments: Side-by-Side

The distinction between parameters and arguments is one of the most commonly confused concepts for beginners. This comparison clarifies the difference with concrete examples.

Parameter (Definition Time)

A **parameter** is the variable name in the `def` line. It is a placeholder – it has no value until the function is called.

```
def add(a, b): # a and b are parameters
    return a + b
```

Here, `a` and `b` are parameters. They do not hold any specific value yet.

Argument (Call Time)

An **argument** is the actual value passed when calling the function. It is assigned to the corresponding parameter.

```
result = add(3, 7) # 3 and 7 are arguments
```

Here, `3` is assigned to `a` and `7` is assigned to `b`. The function returns `10`.



Scope: Local vs. Global Variables

Local Scope

Variables created inside a function. They are **invisible** outside the function. Each function call creates a fresh set of local variables. When the function returns, they are destroyed.

```
def f():  
    x = 5 # local to f()  
    print(x)  
f()  
# print(x) # NameError!
```

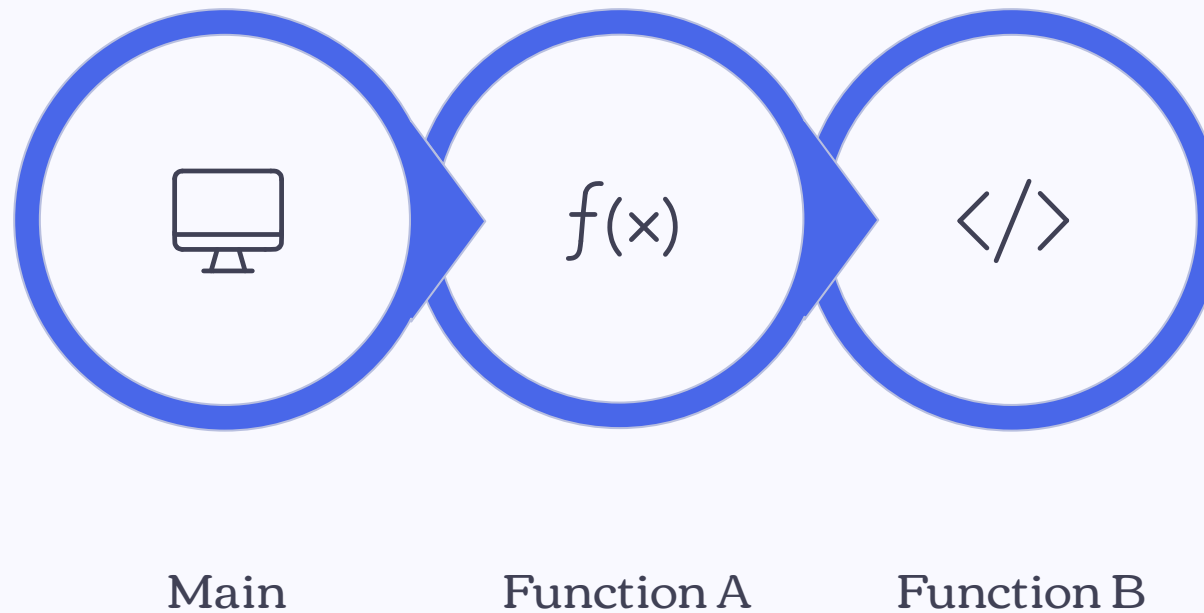
Global Scope

Variables created at the top level of the script. They are **readable** inside functions. To *modify* a global variable inside a function, you must declare it with the `global` keyword.

```
count = 0 # global  
def increment():  
    global count  
    count += 1  
increment()  
print(count) # 1
```

The Call Stack: How Functions Execute

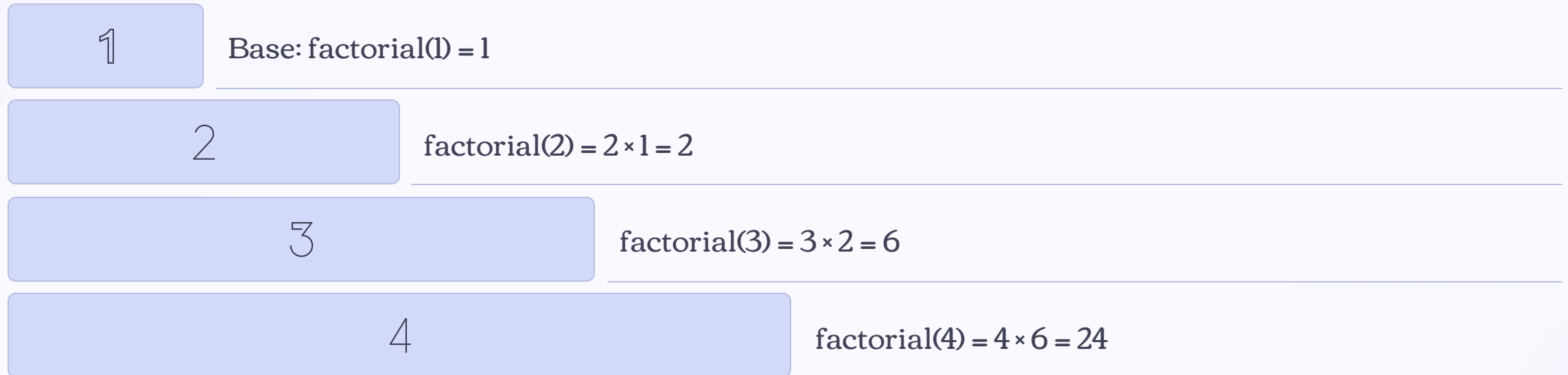
When a function is called, Python pushes a new **stack frame** onto the call stack. This frame holds the function's local variables and tracks where execution should return when the function finishes. Understanding the call stack is key to understanding both regular function calls and recursion.



In recursion, each recursive call adds another frame to the stack. Python has a default recursion limit (typically 1000 frames). Exceeding this limit raises a `RecursionError`. This is why the base case is critical – it stops new frames from being added.

Recursion: Factorial Trace

Tracing through a recursive call helps build intuition. Here is a step-by-step trace of `factorial(4)`:



The staircase mirrors the call stack: each level calls the one below it, and results multiply back up. The base case at the bottom is what makes the whole structure stable.

Fibonacci Sequence via Recursion

The Fibonacci sequence is another classic recursive problem. Each term is the sum of the two preceding terms: $F(n) = F(n-1) + F(n-2)$, with $F(1) = F(2) = 1$.

Recursive Definition

```
def fibonacci(n):  
    if n <= 2:  
        return 1  
    return fibonacci(n-1) + fibonacci(n-2)
```

Two base cases: $F(1) = 1$ and $F(2) = 1$. Every other term is computed by two recursive calls.

First 8 Terms

1 1 2 3 5 8 13 21

Notice that `fibonacci(n)` makes **two** recursive calls. This creates a binary tree of calls – `fibonacci(5)` alone generates 15 function calls. For large `n`, this grows exponentially.



The naive recursive Fibonacci is **$O(2^n)$** in time complexity. For large inputs, use memoization or an iterative approach.

Comparing Recursion and Iteration

Both recursion and iteration (loops) can solve the same problems. Choosing between them depends on clarity, performance, and the nature of the problem.

Recursion

- Elegant for self-referential problems (factorial, Fibonacci, tree traversal)
- Code closely mirrors the mathematical definition
- Each call uses stack memory – risk of `RecursionError` for deep recursion
- Can be slower due to function call overhead

Iteration

- Uses loops (`for`, `while`) to repeat steps
- Generally faster and uses less memory
- Better for large inputs where recursion depth would be exceeded
- May be less intuitive for inherently recursive problems

For this course, recursion is introduced as a conceptual tool. In practice, always consider whether an iterative solution is more appropriate for performance-critical code.

Functions and Modular Programming

Functions are the foundation of **modular programming** – the practice of dividing a program into independent, reusable units. Each function handles one specific task, making the overall program easier to understand, test, and maintain.

Single Responsibility

Each function should do exactly one thing. A function named `triangle_area()` should compute area – nothing else. This makes functions predictable and easy to test in isolation.

Abstraction

Once a function is written and tested, you can use it without thinking about its internal details. `factorial(10)` just works – you don't need to re-trace the recursion every time.

Composability

Functions can call other functions, building complex behavior from simple parts. `print(square(3) + square(4))` composes three functions in one expression.

Default Values: Design Considerations

Default parameter values are a powerful tool for designing flexible, easy-to-use functions. Here are key rules and best practices:

Rule: Defaults Must Come Last

Parameters with default values must appear **after** all parameters without defaults. `def f(a, b=2)` is valid; `def f(a=1, b)` is a `SyntaxError`.

Best Practice: Choose Sensible Defaults

The default should represent the most common use case. For `power(base, exp=2)`, squaring is the most frequent operation, making `2` the natural default.

Overriding Defaults

Any default can be overridden by explicitly passing an argument. `power(5, 3)` overrides `exp=2` with `3`, computing $5^3 = 125$.

Multiple Return Values: Practical Applications

Returning multiple values is a uniquely Pythonic feature with many practical applications in mathematical programming:



Statistical Functions

Return min, max, mean, and standard deviation together.
Example: `return min(data), max(data), mean(data)`



Geometric Calculations

Return multiple properties of a shape at once. Example: `circle(r)` returns both circumference and area.



Quotient and Remainder

Return both results of integer division. Python's built-in `divmod(a, b)` does exactly this: `return a // b, a % b`

Common Function Errors and How to Fix Them

Understanding common mistakes helps you debug faster and write more robust code from the start.

Missing return

Forgetting `return` means the function returns `None`. Assigning the result to a variable gives `None` instead of the expected value. Always check that your function explicitly returns a value when needed.

NameError (Scope)

Trying to access a local variable outside its function raises `NameError: name 'x' is not defined`. The fix: return the value from the function and use it in the calling scope.

RecursionError

Missing or unreachable base case causes infinite recursion. Python raises `RecursionError: maximum recursion depth exceeded`. Always verify your base case is correct and reachable.

Wrong Argument Count

Calling `triangle_area(5)` when the function requires two arguments raises `TypeError: missing 1 required positional argument`. Count your parameters and arguments carefully.

Exercise 3: maximum(a, b, c)

Write a function that returns the largest of three numbers. This exercise can be solved in multiple ways – here are two clean approaches:

Approach 1: Using built-in max()

```
def maximum(a, b, c):  
    return max(a, b, c)  
  
print(maximum(3, 9, 5)) # 9  
print(maximum(10, 2, 7)) # 10
```

Approach 2: Using if/elif

```
def maximum(a, b, c):  
    if a >= b and a >= c:  
        return a  
    elif b >= c:  
        return b  
    else:  
        return c
```

Output

```
9  
10
```

Both approaches are correct. Approach 1 is more concise and Pythonic. Approach 2 demonstrates explicit conditional logic, which is useful for understanding the underlying comparison process.

This exercise practices: function definition, multiple parameters, conditional logic, and returning a computed value.

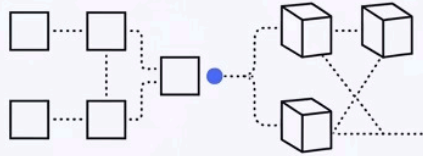
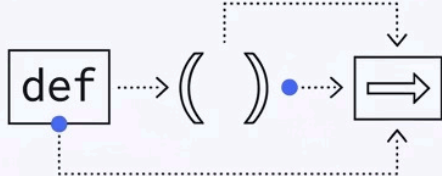
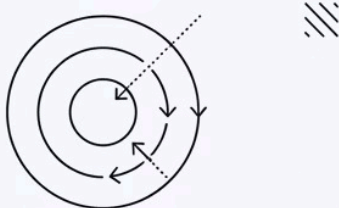
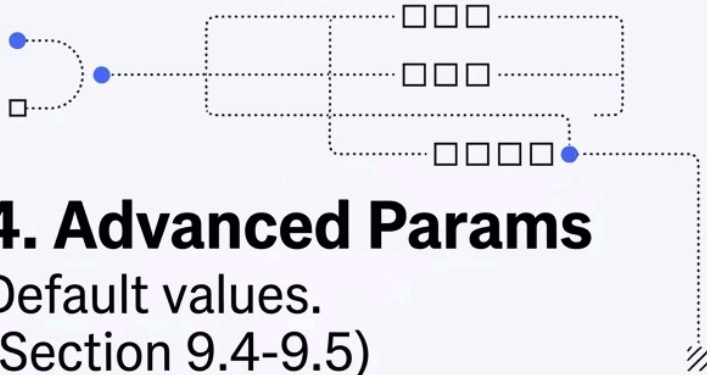
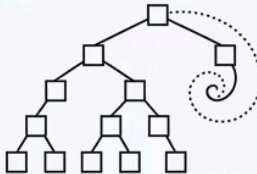
Key Concepts: Quick Reference

A concise reference card for all major concepts introduced in Chapter 9.

Concept	Syntax / Example	Key Point
Define function	<code>def greet(): ...</code>	Use <code>def</code> , indent body
Call function	<code>greet()</code>	Name + parentheses
Parameter	<code>def f(x): ...</code>	Placeholder in definition
Argument	<code>f(5)</code>	Actual value at call time
Return value	<code>return x * x</code>	Sends result to caller
Default value	<code>def power(b, exp=2)</code>	Used when arg is omitted
Multiple return	<code>return a, b, c</code>	Returns a tuple
Local variable	<code>y = 10</code> inside <code>def</code>	Invisible outside function
Recursion	<code>return n * factorial(n-1)</code>	Must have a base case

CLO 4 Alignment

This chapter directly supports **Course Learning Outcome 4 (CLO 4)** of course o4o2231o1. The table below maps each section to the specific learning outcome it develops.

 1. Modular Benefits Code reuse, simplified logic. (Section 9.1)	 2. Functions def name(p): return (Section 9.2-9.3)	 3. Variable Scope Global & local. (Section 9.6)
 4. Advanced Params Default values. (Section 9.4-9.5)	 5. Recursion Math problems. (Section 9.7)	

Students who complete all exercises in this chapter will have demonstrated practical competency in all five dimensions of CLO 4, from basic function definition through to recursive problem-solving for mathematical applications.



Chapter 9 Complete

You have now covered all core concepts of Python functions for mathematical programming.

What You Learned

- Defining and calling functions with `def`
- Parameters, arguments, and `return`
- Default values and multiple return values
- Variable scope: local vs. global
- Recursion with base cases

Next Steps

- Complete all 5 end-of-chapter exercises
- Experiment with writing your own mathematical functions
- Explore Python's built-in functions: `abs()`, `round()`, `divmod()`
- Prepare for Chapter 10: Lists and Arrays



Functions are one of the most important tools in programming. Mastering them unlocks the ability to write clean, powerful, and maintainable code for any mathematical application.